

Decision-Trace Distillation for Reducing LLM Decision Rights in Agentic AI Systems

Oliver Zhou

Lily Li

Abstract

LLM-enabled agentic systems increasingly give large language models operational authority over workflow transitions: interpreting goals, selecting evidence, routing subtasks, choosing tools, evaluating intermediate outputs, writing memory, and deciding whether a process should continue, stop, retry, or escalate. This flexibility is useful when tasks are open-ended or changing, but it also makes consequential workflow behavior harder to reproduce, inspect, audit, and bound. This paper proposes *decision-trace distillation* as a design pattern for reducing unnecessary runtime decision rights of LLM components without removing the adaptability that makes them valuable. During a learning period, the LLM holds broader authority and the system records its decisions together with task context, candidate options, rationales, human feedback, downstream outcomes, overrides, uncertainty signals, and scope or risk tags. During a running period, bounded deterministic or semi-deterministic local components are trained on accepted or outcome-validated traces and receive local decision rights only when confidence, scope, and risk conditions are satisfied. These local components may retrieve precedents, recommend candidate decisions, approve or reject LLM proposals, decide directly above threshold, or participate in a voting procedure. The contribution is not a general claim that smaller models should replace LLMs. It is a governance claim about authority: repeated, well-scoped decisions should become more local, inspectable, and governable, while novel, uncertain, or high-risk cases should remain with the LLM or a human reviewer. The paper specifies the mechanism, relates it to routing, distillation, memory, knowledge compilation, human-in-the-loop governance, and system-level accountability, and proposes an executable evaluation plan for article-structure selection, cover-image theme selection, and economic-data insight selection.

Introduction

A defining feature of many LLM-enabled agentic systems is that the language model is no longer used only to generate a final text response. It often becomes a runtime controller that interprets a request, decomposes a task, selects evidence, calls tools, checks intermediate outputs, updates memory, and decides whether the workflow should continue, stop, retry, or escalate. Surveys of LLM-based agents describe this shift through architectural modules such as profiling, memory, planning, action, perception, reasoning, and execution (L. Wang et al. 2023; Lamo Castrillo et al. 2025). The important design consequence is that a stochastic model can become the component that mediates between open-ended human goals and structured computational action.

This paper describes that mediation as an allocation of *decision rights*. Following Zhou and Li (2026), the term *decision rights* means operational authority over consequential process functions: who selects evidence, who reasons over context, who chooses the next action, who validates an output, who writes to memory, and who determines whether a workflow state should change. The term does not imply moral or legal rights for AI systems. It is a system-design term for locating effective authority among human actors, deterministic

or semi-deterministic system components, and stochastic LLM components.¹ A local learned component is therefore treated as one form of bounded control infrastructure for governance purposes: after decision-trace distillation, a classifier, ranker, retriever, approval model, or policy model may receive part of a decision right that the LLM previously exercised at runtime.

The decision-rights lens matters because agentic AI design is not only a question of which modules are present. Two systems can both contain retrieval, memory, tool use, and human review, yet differ sharply in who actually decides. In one system, the LLM may generate the retrieval query, choose the source, summarize the evidence, select the answer, and write the result to memory. In another, deterministic retrieval, schemas, validators, curated memory, local classifiers, and human approval may constrain those same functions. Typological work usefully classifies agentic systems by capabilities such as autonomy, adaptability, contextualization, and normative alignment (Wissuchek and Zschech 2025). Code-inspection work on agent autonomy similarly shows that autonomy becomes governable only when impact and oversight are decomposed into observable design choices (Cihon et al. 2025). A deployed organization therefore needs not only a capability map but also a rights map: which component holds authority at each decision point, what evidence supports that allocation, and what must be logged, calibrated, audited, or escalated.

Broad LLM decision rights are often rational early in system development because the workflow may not yet have stable categories, reliable features, tested thresholds, or known exception paths. Agent surveys emphasize that LLM-based agents gain practical value from flexible planning, memory, tool use, and feedback-sensitive action rather than from a fixed rule tree alone (L. Wang et al. 2023; Lamo Castrillo et al. 2025). This flexibility has a developmental function. The LLM can explore provisional workflow choices, reveal distinctions that designers did not encode in advance, and produce traces from which a more structured representation can later be built. The governance problem arises when this exploratory allocation remains unchanged after the workflow has become routine. At that point, the system may continue to ask a stochastic general-purpose model to make decisions that have become repetitive, local, and learnable.

This paper proposes *decision-trace distillation* as a mechanism for shifting such recurrent decisions away from the LLM during runtime. During a learning period, the LLM holds broader decision rights and its decisions are logged with context, candidate options, rationale, human feedback, downstream outcomes, overrides, uncertainty indicators, and version metadata. During a running period, a local classifier, ranker, retrieval model, policy model, or approval gate is trained on these traces. The local component receives a bounded right only when its prediction is confident, the case falls inside its approved scope, and the risk level permits local action. Otherwise, the case is deferred to the LLM, a human reviewer, or a hybrid procedure.

The proposal does not reject LLMs. It changes their role. The LLM remains a teacher, an explorer of novel cases, a fallback decision maker, and a participant in hybrid decision procedures. The local component becomes an inspectable memory-based decision component for familiar cases. The central question is therefore not whether an LLM can perform a task in isolation. It is: *which component should hold which decision right for this decision point, under which confidence, scope, and risk conditions?*

¹This paper uses *deterministic component* broadly to include conventional software, rule engines, schemas, calculators, databases, retrievers, bounded routers, workflow engines, validators, knowledge graphs, ontologies, policy constraints, and bounded local learned components when their input interface, version, scope, thresholds, and outputs are observable and controllable. For the fuller distinction between deterministic infrastructure, semi-deterministic control layers, and the LLM component as model-service output at the system boundary, see Zhou and Li (2026).

The paper makes three claims. First, decision traces are useful because they preserve not only what the LLM decided, but also the context, options, feedback, and outcome evidence needed to judge whether that decision should be reused. Second, distillation should be treated as a transfer of bounded authority rather than as a general replacement of the LLM. Third, evaluation should measure decision-share movement, override risk, calibration, deferral, and traceability, not only final task performance.

The rest of the paper develops this argument in six steps. The next section explains why broad LLM decision rights are useful during exploration but costly when routine decisions remain stochastic and hard to audit. The mechanism section then defines decision traces, the learning-period/running-period distinction, graded modes of local decision rights, thresholding, and relearning. The use-case section shows how the pattern applies to article-structure selection, cover-image theme selection, and economic-data insight selection. The evaluation section translates the proposal into testable system variants, metrics, and monitoring criteria. The final sections summarize the contributions, limitations, and governance implications of treating decision authority as an explicit design variable.

Background and motivation

LLM decision rights as a source of adaptability

LLMs are useful in agentic systems because they provide a general semantic interface between unstructured human goals and structured computational action. A user may express an ambiguous goal in natural language, while the system must translate that goal into retrieval choices, workflow steps, tool calls, intermediate checks, and output formats. Unlike a conventional rule engine, an LLM can interpret new contexts without requiring a separately engineered decision tree for every situation. This is why LLM agents are frequently designed as flexible controllers rather than as narrow text generators: their architecture combines language understanding with memory, planning, action modules, and feedback loops ([L. Wang et al. 2023](#); [Lamo Castrillo et al. 2025](#)).

This flexibility is especially valuable when task definitions change frequently. In a content-production workflow, the same agent may need to select the structure of a thought-leadership article, choose the tone of a social post, select the theme of a cover image, or frame an economic-data insight. In an operational workflow, the agent may need to route requests among tools, decide whether a case requires human approval, or choose which prompt strategy should be used. Agentic AI typologies capture this breadth by treating agency as a spectrum of interactivity, autonomy, adaptability, and normative alignment rather than as a binary property ([Wissuchek and Zschech 2025](#)). The same point appears in engineering-oriented agent work that contrasts fixed workflows with adaptive agents: as contextual conditions and goals change, a controller that can interpret context, retrieve memory, plan, and revise actions becomes useful precisely because the correct branch is not always known in advance ([Lamo Castrillo et al. 2025](#)).

The same flexibility can also help a system discover its own future structure. When categories are not yet known, the LLM can make provisional decisions and generate rationales that reveal which distinctions matter in practice. These early decisions are not ground truth. Their value is exploratory: they show which features the LLM attends to, where humans override it, which outcomes count as acceptable, and which cases remain too novel or risky for automation. This exploratory logic is consistent with work that turns experience,

feedback, and action trajectories into reusable future guidance. Experience Notebook stores task-specific lessons after observing feedback on training examples, Agent KB converts heterogeneous execution traces into structured reusable experience units, and LLM-Explorer uses recent action–reward histories to adapt future exploration (Sun et al. 2023; X. Tang et al. 2025; Hao et al. 2025). Decision-trace distillation applies the same general idea to decision authority: the exploratory period is valuable only if its traces become evidence for deciding which future rights can safely be localized.

The governance cost of broad LLM decision rights

The cost of broad LLM decision rights is not limited to API fees or latency. The deeper cost is that important workflow transitions become harder to reproduce, inspect, and contest. The same input can produce different decisions across prompts, model versions, decoding settings, provider-side changes, retrieval order, or hidden safety layers. Work on LLM output drift in financial workflows shows why this matters operationally: even constrained settings can experience variations that undermine audit reproducibility, and mitigation requires deterministic decoding, fixed context, validation, and logging (Khatchadourian and Franco 2025). Work distinguishing bitwise determinism, distributional reproducibility, and semantic stability makes a related point: LLM behavior is better understood as a distribution over possible completions than as a fixed prompt-to-string function (Joshi et al. 2026). Even when an LLM produces a fluent rationale, that rationale should not automatically be treated as a faithful explanation of the decision process, because decision prediction, rationale generation, explanatory hypothesis generation, and decision explanation require different evidential standards (W. Wang 2026).

Agentic systems amplify this problem because an early decision can affect later retrieval, tool use, memory writing, action execution, and output evaluation. Evidence from autonomous-agent failure analysis shows that planning, execution, and response-generation failures can arise at different workflow stages and then propagate through later stages (Lu, Li, and Huo 2025). Safety and security work makes the same point from a risk perspective: harms in agentic systems can emerge from compositional interactions among model outputs, tool calls, memory, retrieval, APIs, user inputs, and orchestration layers (Ghosh et al. 2025). The governance issue is therefore not merely whether the model is accurate at one step. It is whether the system can identify which decision point caused a later operational outcome and which component was authorized to make that decision.

A rights-aware interpretation is that an LLM should not automatically retain authority simply because it can produce a plausible decision. In some cases, a deterministic validator, a local learned component, a retrieval system, or a human reviewer should hold part of the relevant right. Human-in-the-loop designs and calibrated escalation systems show that trust and automation can be made conditional on uncertainty, cost, and task risk (Bachar et al. 2026; Kandikatla and Radeljic 2025). Decision-trace distillation extends that idea from one-off escalation to workflow learning: once enough local traces exist, some repeated decisions can be shifted to bounded non-LLM components with explicit thresholds and audit trails.

The overview figure (Figure 1) summarizes the shift. The left side represents a common default in which the LLM holds many runtime rights at once: it selects evidence, routes tasks, chooses tools, evaluates output, writes memory, and decides whether to stop, retry, or escalate. The right side represents the proposed design after decision-trace distillation. Routine cases are handled by bounded local components; novel or uncertain

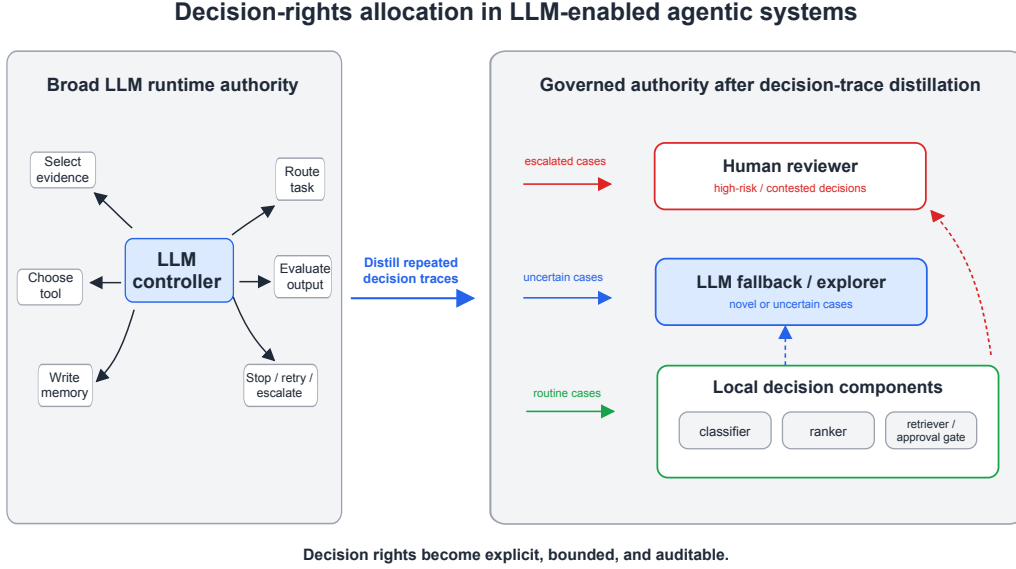


Figure 1: Decision-rights allocation in LLM-enabled agentic systems.

cases remain with the LLM; and high-risk or contested decisions are escalated to human review. The point is not that local components are always superior. The point is that decision rights should become explicit, bounded, and auditable.

Relation to routing, distillation, memory, and knowledge compilation

Several research streams address adjacent parts of this problem. LLM routing studies how to choose among models or components with different cost, latency, and quality profiles (Varangot-Reille et al. 2025). This matters because a router already holds a decision right: it decides which component acts next. Decision-trace distillation extends the routing logic beyond model selection. The routed object may be any repeated workflow decision, including evidence selection, memory writing, approval, or escalation.

Model compression and knowledge distillation study how the behavior of large or ensemble models can be transferred into smaller deployable models (Buciluă, Caruana, and Niculescu-Mizil 2006; Hinton, Vinyals, and Dean 2015). Recent LLM work also shows that teacher–student cooperation benefits from adaptive trust rather than fixed overreliance on the teacher (Ok, Ryu, and Lee 2024). Decision-trace distillation borrows the teacher–student intuition, but the difference is not only that the distilled object is narrower. In conventional distillation, decision authority usually remains inside a model-to-model relation: a smaller model imitates, consults, accelerates, or approximates a larger model. In decision-trace distillation, the governance relation changes because part of a workflow decision right moves from a stochastic LLM component to bounded deterministic or semi-deterministic local infrastructure. The student artifact may be a classifier, ranker, retriever, calibrated approval gate, or policy model, but its role is defined by the right it receives at a specific decision point, phase, and risk scope rather than by its ability to reproduce the LLM’s general semantic competence.

Memory research matters because the local component is trained from past decisions. Agent memory is

often framed as context supplied back to the LLM, but decision-trace distillation treats memory as a source of reusable decision evidence. This is why the relevant memory unit is not merely a text chunk. It must preserve the decision point, candidate set, final accepted decision, feedback, provenance, scope, and outcome signals needed to decide whether a precedent should be reused. Experience Notebook and Agent KB support this view by converting feedback or execution traces into reusable experience structures (Sun et al. 2023; X. Tang et al. 2025). The proposal also connects to knowledge compilation and symbolic grounding work: prompt-compilation workflows externalize quality standards into versioned, testable artifacts, and hybrid expert-system work converts uncertain LLM output into inspectable symbolic knowledge bases that can be validated and reused (Susnjak 2025; Garrido-Merchán and Puente 2025). Decision-trace distillation applies this compilation logic to workflow authority rather than only to prompts, knowledge bases, or model outputs.

This scoped authority-transfer view also distinguishes decision-trace distillation from purely resource-driven local-model deployment. Smaller models can be attractive under fixed budgets when reliable selection or verification is available, but the benefit weakens when the system cannot choose among candidate outputs or validate them (Hassid et al. 2024). Small specialized models can also be effective for narrow checking tasks when training data and benchmarks are constructed around the right evidence relation (L. Tang, Laban, and Durrett 2024). These results support a scoped interpretation of localization: local models are useful when the decision point, candidate set, evidence relation, and escalation rule are explicit.

Proposed mechanism

What is distilled?

The object of distillation is not the LLM’s general linguistic competence. It is a decision trace at a repeated decision point. A decision trace records the relationship among a task context, candidate options, an LLM decision, a rationale or structured explanation, the final accepted decision, and later evidence about whether that decision was useful. In other words, it captures how an initially stochastic decision became accepted, revised, rejected, or escalated inside a workflow.

A decision trace is related to, but narrower than, a full agent trajectory. Agent evaluation work treats trajectories, files, artifacts, and intermediate actions as evidence for judging whether an agent solved a task (Zhuge et al. 2024; Lù et al. 2025). Workflow-abstraction work similarly turns execution traces into structured workflows, and evaluation-agent work audits intermediate decisions, reasoning traces, and outcomes rather than only final metrics (Liu et al. 2026; Du et al. 2026). Decision-trace distillation uses the same evidential orientation but selects a narrower unit: the repeated decision point where a right might be transferred from the LLM to a bounded local component.

This narrower unit determines what the trace must record. Agent evaluation requires trajectory and process records rather than only final-output scores (Yehudai et al. 2025; Zhuge et al. 2024), and system-level evaluation must treat the full agent scaffold—including framework, orchestration, tools, tracing, and evaluation logic—as part of the unit of analysis (Emde et al. 2026). Because generated rationales are not necessarily faithful explanations, the trace should separate the LLM’s stated rationale from accepted decisions and outcome evidence (W. Wang 2026). The purpose of the fields below is therefore not to archive everything an

agent did, but to preserve the minimum evidence needed to decide whether a future local component may safely receive the same right.

A minimal trace should include the following fields:

Field	Purpose
Decision point	Identifies the workflow node, such as article-structure selection, source routing, memory writing, or escalation.
Input context	Stores task description, user goal, source material, metadata, constraints, and relevant history.
Candidate options	Records the finite or semi-finite choice set available at the decision point.
LLM decision	Stores the model’s selected option and any structured output.
LLM rationale	Provides a human-readable account, treated as a useful artifact but not as conclusive explanation.
Final accepted decision	Stores the human-accepted, production-accepted, or outcome-validated decision.
Feedback and override	Records approvals, revisions, rejections, escalations, and reasons for intervention.
Outcome signal	Stores downstream metrics such as revision burden, user acceptance, task success, or risk flags.
Scope and risk tags	Marks whether the case is routine, novel, high-risk, regulated, or outside the local model’s allowed domain.
Version and provenance	Records prompt version, model version, tool state, data source, local-model version, and timestamp.

Each field has an analytical function. The decision point and candidate options define the right being transferred. The input context and scope tags define where that right may be exercised. The final accepted decision, feedback, and outcome signal separate reusable supervision from raw LLM output. The rationale is retained for inspection but is not treated as proof of the decision mechanism. Version and provenance records make later audit possible when a decision is challenged or when model behavior changes.

The central design choice is to train the local model on accepted or outcome-validated decisions rather than on raw LLM outputs alone. If early LLM decisions are simply copied, the system may crystallize LLM errors. Human feedback, overrides, and downstream outcomes should therefore become part of the supervision signal whenever they are available. The target is not “what the LLM said” but “what the workflow eventually accepted, and under what conditions.”

Learning period and running period

The mechanism has two phases. During the *learning period*, the LLM receives broader decision rights because the workflow has not yet accumulated enough local experience. It observes inputs, selects options, explains decisions, and receives feedback from humans or downstream outcomes. The system logs these episodes as decision traces. This period is exploratory: it allows the system to learn which categories, features, thresholds, and exception patterns matter.

During the *running period*, a bounded local component is trained on the accumulated traces. When a new case arrives, the local component predicts a decision, confidence score, and scope assessment. If confidence is high, the case is within the approved scope, and the risk level allows local action, the local component receives the right to act. If not, the decision is deferred to the LLM, a human reviewer, or a hybrid procedure.

The phase distinction is grounded in adjacent work that separates exploratory experience collection from later reuse or local optimization. Experience Notebook first observes feedback on examples and then stores reusable task-specific lessons; Agent KB converts heterogeneous execution traces into structured memory for later retrieval; and two-phase local optimization work in compound AI systems separates an exploratory or free phase from a constrained improvement phase (Sun et al. 2023; X. Tang et al. 2025; Chen et al. 2026). Decision-trace distillation adopts the same functional separation but changes the target: the later phase is not merely better prompting or better memory retrieval; it is bounded transfer of a specific decision right.

The overall mechanism can be summarized as follows:

1. Identify a repeated decision point in an agentic workflow.
2. Define the candidate decision set and the minimum information required for that decision.
3. Let the LLM decide during a learning period while logging context, options, rationales, accepted outcomes, feedback, overrides, uncertainty, and version metadata.
4. Convert traces into structured features, embeddings, exemplars, graph records, or retrieval indexes.
5. Train a local classifier, ranker, retrieval model, approval model, or policy model.
6. Calibrate confidence and define the local component’s approved scope.
7. Deploy the local component with thresholds, audit logs, and escalation paths.
8. Monitor override rate, deferral rate, calibration, distribution shift, downstream quality, and decision-share movement.
9. Reopen learning or retraining when the local component becomes stale or when new decision categories emerge.

This phase structure avoids two errors. The system does not give permanent broad authority to the LLM merely because the LLM was useful during exploration. It also does not prematurely replace the LLM before the workflow has accumulated enough local evidence to support a scoped decision rule.

The lifecycle figure (Figure 2) shows the intended division of labor. The LLM acts as teacher and explorer during the learning period, while the local component acts as a bounded runtime component during the running period. Monitoring closes the loop: high override rates, drift, deferral spikes, or declining quality can reopen the learning period. This feedback path is important because localization should be reversible when the decision environment changes.

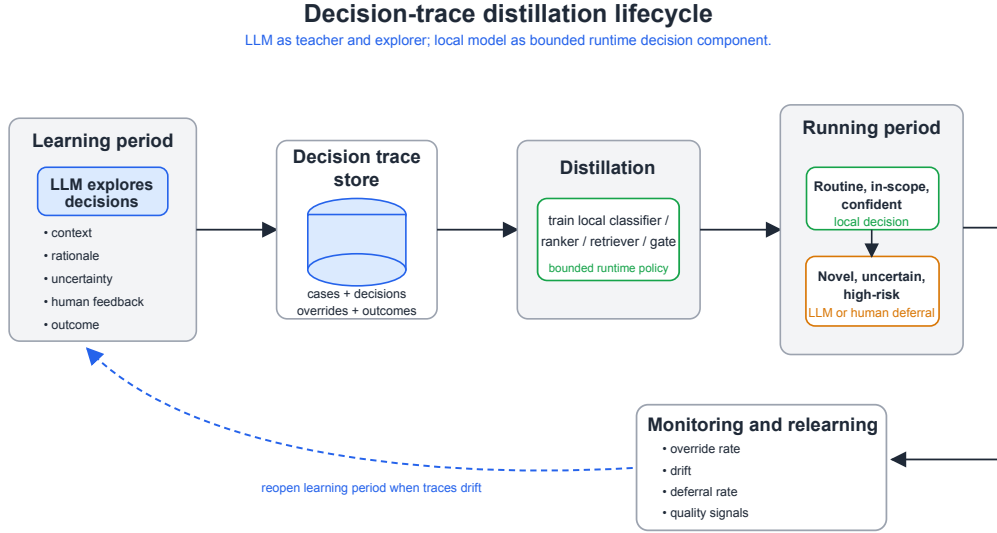


Figure 2: Decision-trace distillation lifecycle.

Five modes of local decision rights

The local component does not need to fully replace the LLM. Decision rights can be reduced gradually. We distinguish five modes:

Mode	Local component's right	LLM's role	Appropriate use
Retrieval support	Right to retrieve similar prior cases	Makes the final decision with precedents visible	Novel, ambiguous, or higher-risk tasks where precedent helps but should not decide
Recommendation	Right to propose one or more likely options	Accepts, revises, rejects, or explains disagreement	Repeated but moderately ambiguous decisions
Approval gate	Right to approve, reject, or flag an LLM proposal	Generates the candidate decision	Brand, policy, safety, or consistency checks
Direct decision	Right to decide above confidence and scope thresholds	Handles deferred cases	Routine, low-risk, finite-choice decisions
Voting member	Right to participate in an ensemble or arbitration process	Votes or supplies reasoning alongside other components	Decisions where diversity and disagreement are useful

This graded design is crucial because reducing LLM decision rights is not the same as replacing the LLM. Replacement is appropriate only when the decision point is routine, bounded, low risk, and empirically validated. In high-risk or weakly labeled domains, the local component may remain a retrieval or approval

component for a long time. In moderately ambiguous domains, it may recommend options but leave final authority with the LLM or a human. The mode should therefore be chosen by the type of right being transferred, not by the availability of a trained model alone.

This mode structure draws support from work that treats autonomy and oversight as graded rather than binary. Agentic AI typologies frame agency as a spectrum; risk-based oversight frameworks map use cases to different forms of human command, validation, and monitoring; controlled-autonomy HITL architectures externalize approval, escalation, and audit logic; and learning-to-escalate systems decide when model outputs should be trusted or deferred (Wissuchek and Zschech 2025; Kandikatla and Radeljic 2025; Cheng and Cheng 2026; Bachar et al. 2026). The contribution here is to apply that graded logic inside a repeated decision point: the local component may receive a small right, such as retrieval, before it receives a stronger right, such as direct action.

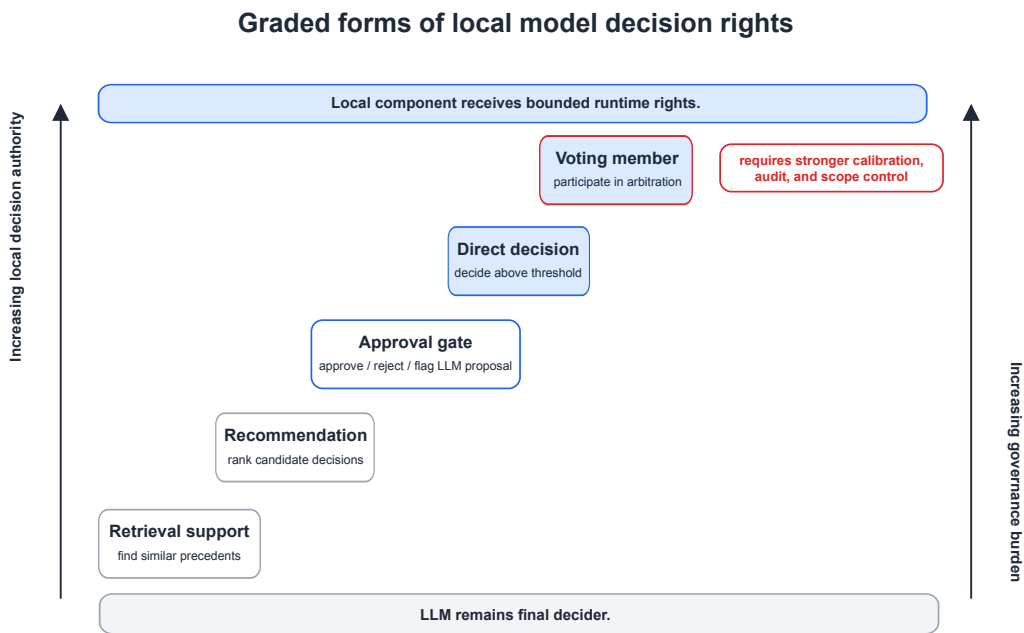


Figure 3: Graded forms of local model decision rights.

The graded-rights figure (Figure 3) makes the governance implication explicit. As local components move from retrieval support to direct decision or voting, they gain more runtime authority and therefore require stronger calibration, audit, and scope control. The same technical artifact can therefore have different governance meanings depending on the right it receives. A classifier that only recommends an option is not equivalent to a classifier that may decide without LLM or human review.

Decision-right thresholding

The decision-right threshold determines when the local component is allowed to act. A simple version is:

$$\text{local_decide}(x) = \begin{cases} 1, & \text{if } p(\hat{y} | x) \geq \tau_d \text{ and } x \in S_d \text{ and } r_d(x) \leq \rho_d, \\ 0, & \text{otherwise.} \end{cases}$$

Here x is the case representation, $\hat{y}$ is the local component’s predicted decision, $p(\hat{y} | x)$ is estimated confidence, τ_d is the confidence threshold for decision point d , S_d is the approved scope of the local component at that decision point, and $r_d(x)$ is a risk score compared with an allowable risk bound ρ_d . Scope constraints prevent the local component from acting outside the distribution for which it was designed. Risk constraints prevent the local component from receiving direct authority even when it is confident but the consequences of error are too high.

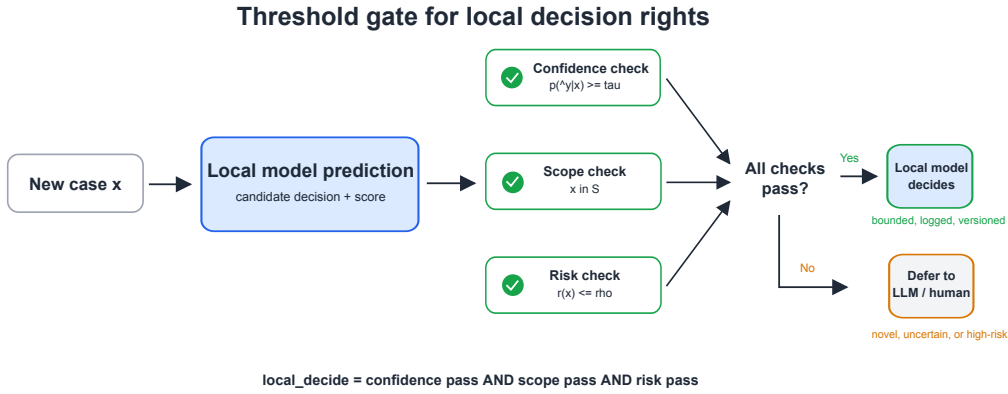


Figure 4: Threshold gate for local decision rights.

The threshold gate figure (Figure 4) translates the equation into an operational rule. A local decision is allowed only when confidence, scope, and risk checks all pass. If any check fails, the system defers. The design therefore treats deferral as a normal governance outcome, not as a failure of the local component. This is consistent with selective prediction and escalation approaches in which automation is valuable only when the system also knows when not to act (Rabanser 2025; Bachar et al. 2026).

The threshold makes decision-right allocation tunable. A higher threshold gives fewer direct rights to the local component and leaves more cases with the LLM or human reviewer. A lower threshold transfers more runtime authority to the local component but may increase override risk. The threshold can differ by decision type. A cover-image theme may allow a lower threshold because the consequence of error is modest. Economic insight selection, credit decisions, safety interventions, or external actions should require stricter thresholds and possibly human approval.

The threshold trade-off figure (Figure 5) is conceptual rather than empirical. It illustrates the expected relationship that must be estimated in deployment: as the confidence threshold rises, the local decision share should generally fall, and override risk should also fall. The candidate operating region is not the point that maximizes local autonomy. It is the region where the system reduces unnecessary LLM decision share while keeping override risk and deferral burden within the organization’s tolerance.

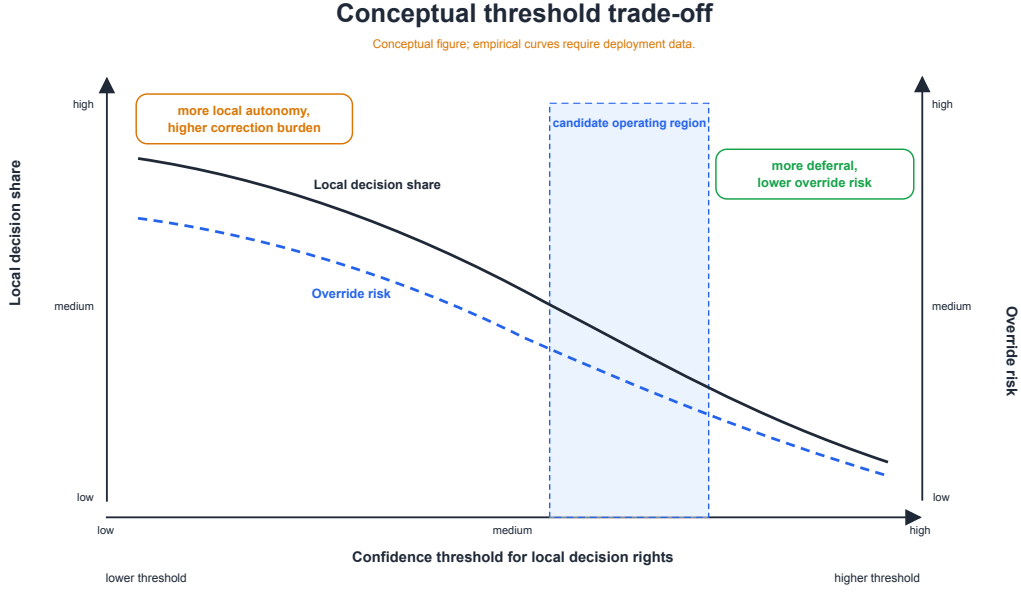


Figure 5: Conceptual threshold trade-off.

Relearning, reconsideration, and governance

A system that reduces LLM decision rights must still preserve adaptability. The local component can become stale when the user population, brand strategy, market environment, task definition, tool set, or model provider changes. Relearning is therefore part of the mechanism rather than an optional maintenance task.

The relearning module monitors deferral frequency, confidence decline, override rate, disagreement with the LLM, human correction patterns, downstream quality, and distribution shift. When these indicators cross a threshold, the system reopens the learning period for that decision point. This is a form of reconsideration: the system asks whether a right that had been localized should remain local, be retrained, be narrowed, or return to the LLM or human reviewer.

This governance loop is consistent with adjacent work that treats deployment as iterative rather than final. Responsible LLM deployment work combines uncertainty routing, human evaluation, robust explanation checks, reviewed samples, and audit trails; Agent KB updates reusable experience from execution traces and failures; and system-level accountability work argues that agentic systems require tracing across perception, reasoning, planning, memory, tool execution, and coordination rather than isolated model explanations (Sachan, Miller, and Nguyen 2025; X. Tang et al. 2025; Zhu et al. 2026). Decision-trace distillation turns this loop into an authority-control mechanism: monitoring is not only for performance repair, but also for deciding whether a localized right should be retained, narrowed, or revoked.

This governance loop should preserve traceability. Local decisions should be logged with the model version, feature representation, confidence, threshold, scope rule, retrieved precedents, and any later override. This supports system-level accountability because the relevant question is not merely why a model emitted a text string, but how a sequence of component decisions produced an operational outcome. It also supports targeted guardrail placement because the system can identify which decision points propagate the most risk

(Ghosh et al. 2025; Zhu et al. 2026).

Use cases

The initial use cases are intentionally modest. The purpose is not to prove that all agentic decisions can be localized. It is to show how decision-trace distillation works for repeated decisions whose input context, candidate set, feedback signal, and risk level can be made explicit.

Article-structure selection

The first use case is selecting the structure of a Cognaptus blog article. In the initial workflow, an LLM reads the topic, target audience, source material, intended message, desired tone, length constraint, and publishing context. It then selects an article structure, such as explanatory essay, problem–solution, trend analysis, case-based argument, data commentary, opinion note, or executive briefing.

During the learning period, each decision is recorded with topic features, audience, source density, desired argument style, tone, final human-selected structure, revision burden, and publication outcome. After enough examples accumulate, a local classifier can predict the preferred structure for familiar cases. At runtime, the classifier may directly choose the structure above threshold. When confidence is lower, it can provide the top candidate structures to the LLM together with similar precedents.

This use case is suitable because the decision is repeated, the output classes are finite, and human feedback is available. It also has measurable downstream signals: whether the selected structure leads to fewer revisions, faster acceptance, or better fit with the intended message. The main risk is over-localization. If the local model learns a narrow house style too strongly, it may suppress useful novelty. The threshold and relearning module should therefore monitor topic novelty and editor overrides.

Cover-image theme selection

The second use case is selecting a cover-image theme for a Cognaptus blog article. The LLM initially chooses among themes such as abstract concept, human collaboration, data visualization, market metaphor, technology interface, industry scene, or minimalist symbolic design. This choice depends on article title, topic, audience, intended emotional tone, source material, and brand memory.

A local model can learn from previous LLM and human decisions. For example, articles about organizational strategy may often use abstract or metaphorical imagery, while data-heavy macroeconomic notes may use chart-based or market-themed imagery. The local model can then select or recommend a theme when the article matches previous patterns. For ambiguous or novel topics, the decision is deferred to the LLM or to a human editor.

This use case separates creative generation from decision control. The local model does not need to generate the image. It only selects the theme, approves whether a proposed theme is consistent with brand memory, or retrieves similar prior design decisions. A generative image model can still create candidate images, while the local component governs part of the decision process.

Economic-data insight selection

The third use case is selecting the main insight from economic data. Compared with the first two use cases, this decision is more difficult and consequential. The input may include time series, metadata, market expectations, revisions, calendar context, historical regimes, related indicators, and domain-specific interpretation. The output may be an insight frame, such as trend continuation, structural break, anomaly, policy implication, sector divergence, cyclical turning point, or risk alert.

For this use case, the local model should not receive full direct decision rights at the beginning. A better first role is retrieval support, recommendation, or voting. It can retrieve similar historical cases, classify the likely insight type, identify whether the current pattern resembles known episodes, and flag when the observation is outside previous experience. The LLM can then use this local precedent as a constraint or reference rather than deciding from scratch.

This use case illustrates the difference between reducing and eliminating LLM decision rights. The LLM may remain valuable for synthesizing macroeconomic context, explaining trade-offs, and generating narrative implications. The local model reduces unchecked authority by anchoring the LLM to precedent, scope, and historical patterns. Over time, if the decision set becomes sufficiently stable and feedback signals become reliable, the system may allow direct local decisions for narrow low-risk subcases while preserving LLM or human review for novel or high-impact interpretations.

Evaluation plan

The empirical evaluation should ask whether decision-trace distillation can reduce runtime LLM decision share while preserving or improving accepted decision quality, stability, interpretability, and governance. This is different from asking whether the local model always outperforms the LLM. A successful hybrid system may preserve most accepted decisions while shifting routine decisions away from the LLM and escalating uncertain cases.

The evaluation should be framed as a comparative system study rather than as a broad causal claim about LLMs or local models. The treatment of interest is a workflow design: whether a repeated decision point is handled by an LLM-only process, a local-only process, a thresholded hybrid process, or a conservative support process. The strongest initial evidence will come from fixed-task comparisons in which each variant receives the same input cases, candidate options, logging schema, and outcome labels. This framing matches the direction of agent-evaluation work that treats performance as an emergent property of the complete agent system rather than the underlying model alone (Yehudai et al. 2025; Emde et al. 2026).

Research questions

The first evaluation round can be organized around four research questions:

1. **Decision-share reduction:** How much of the runtime decision share can be shifted from the LLM to the local model under different thresholds?
2. **Quality preservation:** Does the hybrid system match final human-accepted or production-accepted decisions at a rate comparable to the LLM-only baseline?

3. **Governability:** Are repeated decisions more stable, traceable, and easier to inspect when handled by local components?
4. **Risk-sensitive deferral:** Does the system defer novel, uncertain, or high-risk cases rather than over-confidently localizing them?

These questions align with the broader move in agent evaluation away from final-output scoring toward trajectory, process, artifact, safety, and long-horizon assessment (Yehudai et al. 2025; Zhuge et al. 2024). They also reflect the autonomy problem at the center of this paper: autonomy should be treated as a configurable design variable, not as a direct consequence of model capability.

Data construction

The evaluation dataset should be built at the decision-point level rather than at the whole-workflow level. Each row should represent one decision opportunity, such as selecting an article structure, choosing a cover-image theme, or assigning an economic insight frame. The minimum record should include the input context, candidate option set, LLM-selected option, human-accepted or production-accepted option, override reason if any, downstream quality signal, uncertainty signal, and risk or scope tag.

For article-structure and cover-image use cases, retrospective logs can provide the first dataset. Earlier article topics, LLM recommendations, editor-selected structures, image themes, revision outcomes, and publication decisions can be converted into supervised examples. A chronological split is preferable to a random split because the real deployment problem is whether earlier traces can support later decisions under changing topics and editorial preferences.

For the economic-data insight use case, retrospective evaluation should begin with support roles. Human experts can assess whether retrieved precedents are relevant, whether suggested insight frames are plausible, and whether the system appropriately flags novel or uncertain cases. Only after this support role is validated should the local model receive narrow direct decision rights.

System variants

The evaluation should compare at least four variants:

Variant	Description	Purpose
LLM-only	The LLM makes the decision at each decision point.	Baseline for quality, cost, latency, and stability.
Local-only	The local model decides whenever it has a prediction.	Stress test for over-localization risk.
Hybrid threshold	The local model decides only above threshold and defers other cases.	Main proposed design.

Variant	Description	Purpose
Hybrid support	The local model retrieves, recommends, approves, or votes, but the LLM or human remains final decider.	Conservative design for higher-risk tasks.

The local-only variant is not necessarily a recommended deployment pattern. It is useful because it shows what happens when too many rights are shifted away from the LLM. The hybrid variants are more important for practical design because they reveal the trade-off between local decision share, quality, and override risk.

Experimental protocol

A practical first protocol has six steps. First, choose one decision point and freeze its candidate option set for the initial evaluation period. Second, construct a training set from earlier accepted traces and a validation set for threshold selection. Third, train one or more local components, such as a classifier for finite options, a ranker for top- k recommendations, or a retrieval model for precedent search. Fourth, calibrate confidence on the validation set and define scope rules, including novelty indicators and excluded case types. Fifth, evaluate all system variants on a chronologically later test set. Sixth, repeat the evaluation across a threshold sweep so that the trade-off among local decision share, deferral rate, accepted-decision agreement, and override risk can be observed.

This protocol should avoid three common overstatements. First, matching the LLM’s raw decision is not the main target when human-accepted or outcome-validated decisions are available. Second, high local accuracy is insufficient if the local model acts outside its approved scope. Third, low deferral is not automatically good; for high-risk or novel cases, deferral is the desired behavior.

Metrics

The following metrics should be reported:

Metric	Meaning
Agreement with accepted decision	Whether the system matches the final human-accepted or production-accepted decision.
Downstream quality	Whether the resulting article, image theme, or insight requires fewer revisions or produces better task outcomes.
LLM decision share	Percentage of cases where the LLM makes the final decision.
Local decision share	Percentage of cases where the local model makes the final decision.

Metric	Meaning
Deferral rate	Percentage of cases escalated to the LLM or human reviewer.
Override rate	Percentage of local decisions later corrected by the LLM, human, or downstream evaluation.
Stability	Consistency of decisions across repeated runs on the same input.
Calibration	Relationship between predicted confidence and observed correctness.
Scope violation rate	Frequency with which the local model acts outside its approved domain.
Interpretability	Ability to inspect features, precedents, rules, thresholds, and model versions behind a decision.
Cost and latency	Reduction in model calls, response time, and operational expense.

The central expected output is a threshold curve. As the confidence threshold decreases, the local model receives more direct decision rights, cost and latency may fall, and stability may improve. At the same time, override risk may rise. The desirable threshold is not universal. It depends on task risk, user tolerance, availability of human review, and the cost of mistakes at scale.

Trajectory and artifact evidence should be inspected for contested cases rather than relying only on aggregate metrics. Work on agent judges shows that process-sensitive evaluation can inspect trajectories, files, and artifacts, while work on web-agent trajectory evaluation also warns that both LLM judges and brittle rule-based evaluators can misclassify agent outcomes (Zhuge et al. 2024; Lù et al. 2025). For decision-trace distillation, this means evaluation should retain enough evidence to explain why a local decision was accepted, overridden, or deferred.

Prospective monitoring and relearning criteria

Retrospective evaluation is only the first step. A prospective deployment should monitor the same decision point after the local component receives bounded rights. The monitoring dashboard should report local decision share, deferral rate, override rate, calibration drift, category-level error concentration, and changes in the distribution of topics or input features. These metrics should be tracked by decision type rather than only in aggregate, because a stable average can hide deterioration in a newly important category.

Relearning should be triggered when predefined indicators cross governance thresholds. Examples include a sustained increase in human overrides, a rise in high-confidence errors, a decline in calibration, a spike in out-of-scope cases, or the appearance of new candidate options. When a trigger fires, the system should either reopen the learning period, narrow the local model’s scope, raise the decision threshold, or return the decision right to the LLM or human reviewer until updated traces support relocalization.

Contributions

This paper makes three contributions.

First, it reframes model distillation as a decision-rights allocation mechanism. Traditional distillation and model compression transfer predictive behavior from a larger teacher into a smaller student (Buciluă, Caruana, and Niculescu-Mizil 2006; Hinton, Vinyals, and Dean 2015). Decision-trace distillation transfers not only a narrower predictive target, but also a bounded portion of workflow authority: the authority to make a repeated decision under explicit scope, confidence, and risk constraints.

Second, it specifies a graded design pattern for reducing LLM runtime authority. The local model can retrieve precedents, recommend options, approve LLM proposals, decide directly above threshold, or participate in voting. This graded view prevents the false choice between full LLM autonomy and full deterministic replacement. It also connects agentic AI governance to the practical design of thresholds, deferral, audit logs, and retraining triggers.

Third, it links local memory, knowledge compilation, and system-level accountability. Agent memory is often framed as a way to improve future LLM outputs. This paper adds another interpretation: when memory is converted into a classifier, ranker, approval gate, or retrieval model, it becomes a local decision component. The result is a more inspectable agentic system in which familiar decisions can be governed locally and unfamiliar decisions can remain adaptive.

Limitations

This paper is an initial design proposal and does not claim that local models should replace LLMs in all agentic decisions. Some decisions are rare, open-ended, creative, socially sensitive, or highly context-dependent. In those cases, the LLM may remain the most appropriate semantic reasoner, and the local model should only retrieve precedents, provide checks, or assist human review.

The proposal also depends on the quality of logged decisions. If the learning period records poor LLM decisions, biased outputs, weak rationales, or unrepresentative human feedback, the local model may reproduce those problems. The most important training target is therefore not the raw LLM decision but the final accepted or outcome-validated decision. Human corrections, downstream performance, and override reasons should be treated as first-class data.

Another limitation is distribution shift. A local model trained on past decisions may become outdated when the brand strategy, audience, market environment, tool set, model provider, or task definition changes. Re-learning and reconsideration are therefore essential. A system should be able to narrow, suspend, retrain, or revoke local decision rights when performance indicators deteriorate.

A further limitation is measurement. Effective decision rights may differ from formal rights. A human may formally approve an output but lack enough information to challenge it. A local model may formally only recommend an option but effectively anchor the LLM’s later choice. An LLM may formally only generate rationales but shape the option set in ways that determine the final decision. Future empirical work should estimate effective rights from logs, interface behavior, routing records, memory writes, approval patterns,

and failure cases.

Finally, the pattern may create new governance responsibilities. Local models can appear more stable and transparent than LLMs, but they still require calibration, monitoring, documentation, and scope control. A poorly calibrated local model can silently receive too much authority. Decision-trace distillation should therefore be deployed as a governed workflow pattern, not merely as a technical optimization.

Conclusion

Agentic AI systems gain adaptability by giving LLMs broad decision rights. However, broad LLM decision rights also create practical problems of stability, reproducibility, transparency, and accountability. This paper proposed decision-trace distillation as a design pattern for reducing unnecessary LLM runtime authority while preserving LLM adaptability for novel, uncertain, and high-risk cases.

The mechanism uses the LLM heavily during a learning period, records decision traces, and then trains bounded deterministic or semi-deterministic local components for recurrent decisions. The local component may retrieve, recommend, approve, decide, or vote depending on confidence, scope, and risk. The LLM becomes a teacher, fallback decision maker, and explorer of novel cases rather than the permanent holder of all workflow authority.

The broader research agenda is to make decision authority an explicit and configurable property of agentic systems. Instead of implicitly giving the most general model the most authority, designers should ask which party should hold which right at each decision point. Decision-trace distillation offers one practical answer: let LLMs help discover local regularities, then shift repeated decisions into more bounded, auditable, and governable components when the evidence supports doing so.

References

- Bachar, Or, Or Levi, Sardhendu Mishra, Adi Levi, Manpreet Singh Minhas, Justin Miller, Omer Ben-Porat, Eilon Sheerit, and Jonathan Morra. 2026. “LLM Performance Predictors: Learning When to Escalate in Hybrid Human-AI Moderation Systems.” <https://arxiv.org/abs/2601.07006v1>.
- Buciluă, Cristian, Rich Caruana, and Alexandru Niculescu-Mizil. 2006. “Model Compression.” *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 535–41. <https://doi.org/10.1145/1150402.1150464>.
- Chen, Minghui, Wenlong Deng, James Zou, Han Yu, and Xiaoxiao Li. 2026. “Textual Equilibrium Propagation for Deep Compound AI Systems.” <https://arxiv.org/abs/2601.21064v3>.
- Cheng, Edward, and Jeshua Cheng. 2026. “A Decoupled Human-in-the-Loop System for Controlled Autonomy in Agentic Workflows.” <https://arxiv.org/abs/2604.23049v1>.
- Cihon, Peter, Merlin Stein, Gagan Bansal, Sam Manning, and Kevin Xu. 2025. “Measuring AI Agent Autonomy: Towards a Scalable Approach with Code Inspection.” <https://arxiv.org/abs/2502.15212v1>.
- Du, Gaoyuan, Amit Ahlawat, Xiaoyang Liu, and Jing Wu. 2026. “A Framework for Assessing AI Agent Decisions and Outcomes in AutoML Pipelines.” <https://arxiv.org/abs/2602.22442v2>.

- Emde, Cornelius, Alexander Rubinstein, Anmol Goel, Ahmed Heakl, Sangdoo Yun, Seong Joon Oh, and Martin Gubri. 2026. “MASEval: Extending Multi-Agent Evaluation from Models to Systems.” <https://arxiv.org/abs/2603.08835v1>.
- Garrido-Merchán, Eduardo C., and Cristina Puente. 2025. “GOF AI Meets Generative AI: Development of Expert Systems by Means of Large Language Models.” <https://arxiv.org/abs/2507.13550v2>.
- Ghosh, Shaona, Barnaby Simkin, Kyriacos Shiarlis, Soumili Nandi, Dan Zhao, Matthew Fiedler, Julia Bazinska, et al. 2025. “A Safety and Security Framework for Real-World Agentic Systems.” <https://arxiv.org/abs/2511.21990v1>.
- Hao, Qian Yue, Yiwen Song, Qingmin Liao, Jian Yuan, and Yong Li. 2025. “LLM-Explorer: A Plug-in Reinforcement Learning Policy Exploration Enhancement Driven by Large Language Models.” <https://arxiv.org/abs/2505.15293v2>.
- Hassid, Michael, Tal Remez, Jonas Gehring, Roy Schwartz, and Yossi Adi. 2024. “The Larger the Better? Improved LLM Code-Generation via Budget Reallocation.” <https://arxiv.org/abs/2404.00725v2>.
- Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. “Distilling the Knowledge in a Neural Network.” <https://arxiv.org/abs/1503.02531v1>.
- Joshi, Tanmay, Shourya Aggarwal, Anusa Saha, Aadi Pandey, Shreyash Dhoot, Vighnesh Rai, Raxit Goswami, Aman Chadha, Vinija Jain, and Amitava Das. 2026. “Stochastic CHAOS: Why Deterministic Inference Kills, and Distributional Variability Is the Heartbeat of Artificial Cognition.” <https://arxiv.org/abs/2601.07239v1>.
- Kandikatta, Laxmiraju, and Branislav Radeljic. 2025. “AI and Human Oversight: A Risk-Based Framework for Alignment.” <https://arxiv.org/abs/2510.09090v1>.
- Khatchadourian, Raffi, and Rolando Franco. 2025. “LLM Output Drift: Cross-Provider Validation & Mitigation for Financial Workflows.” <https://arxiv.org/abs/2511.07585v1>.
- Lamo Castrillo, Victor de, Habtom Kahsay Gidey, Alexander Lenz, and Alois Knoll. 2025. “Fundamentals of Building Autonomous LLM Agents.” <https://arxiv.org/abs/2510.09244v1>.
- Liu, Yihao, Ziyun Zhang, Zile He, and Huaqian Cai. 2026. “FlowMind: Execute-Summarize for Structured Workflow Generation from LLM Reasoning.” <https://arxiv.org/abs/2602.11782v1>.
- Lu, Ruofan, Yichen Li, and Yintong Huo. 2025. “Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks.” <https://arxiv.org/abs/2508.13143v1>.
- Lù, Xing Han, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. 2025. “AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories.” <https://arxiv.org/abs/2504.08942v2>.
- Ok, Hyunjong, Jegwang Ryu, and Jaeho Lee. 2024. “Decoding with Limited Teacher Supervision Requires Understanding When to Trust the Teacher.” <https://arxiv.org/abs/2406.18002v2>.
- Rabanser, Stephan. 2025. “Uncertainty-Driven Reliability: Selective Prediction and Trustworthy Deployment in Modern Machine Learning.” <https://arxiv.org/abs/2508.07556v2>.
- Sachan, Swati, Theo Miller, and Mai Phuong Nguyen. 2025. “Responsible LLM Deployment for High-Stake Decisions by Decentralized Technologies and Human-AI Interactions.” <https://doi.org/10.1109/ICHMS65439.2025.11154208>.
- Sun, Wangtao, Xuanqing Yu, Shizhu He, Jun Zhao, and Kang Liu. 2023. “ExpNote: Black-Box Large Language Models Are Better Task Solvers with Experience Notebook.” <https://arxiv.org/abs/2311.07032v2>.

- Susnjak, Teo. 2025. “Compiling Prompts, Not Crafting Them: A Reproducible Workflow for AI-Assisted Evidence Synthesis.” <https://arxiv.org/abs/2509.00038v1>.
- Tang, Liyan, Philippe Laban, and Greg Durrett. 2024. “MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents.” <https://arxiv.org/abs/2404.10774v2>.
- Tang, Xiangru, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, et al. 2025. “Agent KB: Leveraging Cross-Domain Experience for Agentic Problem Solving.” <https://arxiv.org/abs/2507.06229v5>.
- Varangot-Reille, Clovis, Christophe Bouvard, Antoine Gourru, Mathieu Ciancone, Marion Schaeffer, and François Jacquenet. 2025. “Doing More with Less: A Survey on Routing Strategies for Resource Optimisation in Large Language Model-Based Systems.” <https://arxiv.org/abs/2502.00409v3>.
- Wang, Lei, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, et al. 2023. “A Survey on Large Language Model Based Autonomous Agents.” <https://doi.org/10.1007/s11704-024-40231-1>.
- Wang, Wenshuo. 2026. “LLMs Should Not yet Be Credited with Decision Explanation.” <https://arxiv.org/abs/2605.01164v1>.
- Wissuchek, Christopher, and Patrick Zschech. 2025. “Exploring Agentic Artificial Intelligence Systems: Towards a Typological Framework.” <https://arxiv.org/abs/2508.00844v1>.
- Yehudai, Asaf, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2025. “Survey on Evaluation of LLM-Based Agents.” <https://arxiv.org/abs/2503.16416v1>.
- Zhou, Oliver, and Lily Li. 2026. “Decision-Rights Allocation: A Governance-Centric Perspective on LLM-Enabled Agentic Systems.” Quantum Capital AI Ltd. https://oliverlds.github.io/files/decision_rights_allocation_concept.pdf.
- Zhu, Judy, Dhari Gandhi, Himanshu Joshi, Ahmad Rezaie Mianroodi, Sedef Akinli Kocak, and Dhanesh Ramachandran. 2026. “Interpreting Agentic Systems: Beyond Model Explanations to System-Level Accountability.” <https://arxiv.org/abs/2601.17168v1>.
- Zhuge, Mingchen, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, et al. 2024. “Agent-as-a-Judge: Evaluate Agents with Agents.” <https://arxiv.org/abs/2410.10934v2>.